



1. DATOS GENERALES DE LA ASIGNATURA

ASIGNATURA: Gestión del software						
Créditos:	Tipo			Ciclo		
	Teórica	Teórica/práctica	Práctica	Fundamentación	Profundización	Innovación
Modalidad	Virtual					
Semestre:	7				Horas totales de trabajo: 144	

2. SISTEMA DE EVALUACIÓN

	Actividad de aprendizaje	Nombre de la Actividad	Porcentaje evaluativo
Semana 1	Quiz Diagnóstico	Cuestionario evaluativo inicial	0 %
Semana 1	AA1(unid 1)	Foro de Discusión: Resolución de Conflictos en Git	5 %
Semana 2	AA2 (unid 1)	Taller: Mapa Mental sobre Herramientas de Control de Versiones (Git y SVN)	15 %
Semana 3	AA1(unid 2)	Actividad: Elaboración de un Mapa Conceptual sobre la Gestión de Cambios en el Desarrollo de Software	5 %
Semana 4	AA2 (unid 2)	Taller Grupal – Línea Base Software de Restaurante	20 %
Semana 5	AA1(unid 3)	Estudio de Caso: Integración Continua en un Proyecto de Software	5 %
Semana 6	AA2 (unid 3)	ABP: Proyecto Integrador – Construcción de un Pipeline para un Proyecto de Software	20 %
Semana 7	Examen final	Cuestionario evaluativo final	30 %
Semana 8	CIERRE DEL CURSO		
TOTAL EVALUACIÓN			100 %

3. DESCRIPCIÓN DE LAS ACTIVIDADES

3.1 Unidad

Unidad 1:Control de versiones

3.1.2 Nombre actividad

- AA1. Foro de Discusión: Resolución de Conflictos en Git
- AA2. Taller: Mapa Mental sobre Herramientas de Control de Versiones (Git y SVN)

3.1.3 Tipo de actividad

- AA1: Foro
- AA2: Taller(Mapa Mental)

3.1.4 Desarrollo actividad

AA1. Foro de Discusión: Resolución de Conflictos en Git

Descripción de la Actividad:

En esta actividad, los estudiantes analizarán y debatirán las estrategias más eficaces para la resolución de conflictos en Git, una herramienta esencial en la gestión de versiones durante el desarrollo de software. Los conflictos en Git surgen habitualmente cuando varios desarrolladores realizan cambios simultáneos en los mismos archivos o líneas de código. Conocer las causas comunes, las mejores prácticas para resolverlos y los métodos preventivos permite mejorar la colaboración en el equipo y mantener la integridad del código. A través del intercambio de ideas y experiencias, se busca fortalecer la comprensión de Git como herramienta clave en la coordinación de equipos de desarrollo.

Preguntas para resolver

1. En el uso de Git, los conflictos pueden aparecer cuando dos desarrolladores editan el mismo archivo simultáneamente. En opinión del estudiante, ¿cuál es la causa más común de estos conflictos y qué estrategia se considera más adecuada para resolverlos? Se espera justificar la respuesta mediante ejemplos o situaciones reales.
2. Existen diversas prácticas que pueden prevenir conflictos al trabajar con Git. ¿Qué combinación de buenas prácticas debería adoptarse en un proyecto colaborativo para minimizar la aparición de conflictos durante las integraciones? ¿Por qué?



Objetivo del Foro

El propósito de este foro es que los estudiantes:

- Reconozcan las causas frecuentes de conflictos al usar Git en proyectos colaborativos.
- Analicen técnicas efectivas para resolver estos conflictos de forma segura y organizada.
- Reflexionen sobre cómo las buenas prácticas en Git pueden prevenir errores y optimizar la colaboración entre desarrolladores.

Instrucciones para la Participación

1. Publicación inicial:
 - Se solicita a cada estudiante responder ambas preguntas utilizando argumentos claros y ejemplos relacionados con el uso real de Git.
 - La publicación inicial debe tener una extensión mínima de 100 palabras y debe ser realizada antes de la fecha límite.
2. Interacción con compañeros:
 - Leer las publicaciones de al menos dos compañeros.
 - Realizar comentarios que aporten valor: pueden complementar ideas, compartir experiencias relacionadas, proponer herramientas alternativas o sugerir mejoras.
 - Cada comentario debe contener un mínimo de 100 palabras.
3. Lenguaje y respeto:
 - Se debe mantener un lenguaje profesional, claro y respetuoso en todas las intervenciones.
 - No se permite copiar contenido de internet o de otros compañeros. Todos los aportes deben ser originales y personales.

Retroalimentación

- El facilitador revisará las participaciones y proporcionará retroalimentación constructiva.
- Se destacarán los análisis bien fundamentados, las soluciones creativas y las interacciones que promuevan un diálogo enriquecedor.
- La retroalimentación se publicará dentro del foro para que todos los participantes puedan beneficiarse.



AA2. Taller: Mapa Mental sobre Herramientas de Control de Versiones (Git y SVN)

Descripción de la actividad

En este taller, los estudiantes desarrollarán un mapa mental que les permita representar de forma visual y estructurada los conceptos clave relacionados con las herramientas de control de versiones, con énfasis en Git y SVN. Esta representación facilitará la comprensión de cómo se gestionan los cambios en el desarrollo de software, se facilita la colaboración entre desarrolladores y se mantiene un historial claro de modificaciones en los proyectos.

Instrucciones para la actividad

1. Tema Principal del Mapa Mental :

Herramientas de control de versiones: Git y SVN

Subtemas sugeridos:

- Conceptos fundamentales
 - ¿Qué es el control de versiones?
 - Ventajas de utilizar sistemas de control de versiones.
- Git
 - Tipos de control: distribuido.
 - Comandos esenciales: git init, git clone, git commit, git push, git pull, git merge, git branch.
 - Estructura de trabajo: staging área, repositorio local y remoto.
 - Manejo de ramas y fusiones.
 - Resolución de conflictos.
- SVN (Subversion)
 - Tipo de control: centralizado.
 - Comandos principales: svn checkout, svn update, svn commit.
 - Estructura de carpetas típica: /trunk, /branches, /tags.
 - Ventajas y limitaciones frente a Git.
- Comparativa Git vs SVN
 - Modelo de control (distribuido vs centralizado).
 - Flexibilidad y escalabilidad.
 - Uso sin conexión.
 - Escenarios recomendados para cada herramienta.
- Buenas prácticas en el uso de herramientas de control
 - Realizar commits frecuentes y descriptivos.



- Evitar conflictos de fusión.
- Uso de ramas temáticas.
- Coordinación en equipos de trabajo.
- ★ Errores comunes a evitar
 - No documentar los cambios.
 - Trabajar directamente en la rama principal.
 - Ignorar conflictos sin revisarlos adecuadamente.
- 2. **Desarrollo del Mapa Mental :**
 - Organizar la información siguiendo una estructura jerárquica, partiendo del tema central hacia subtemas y detalles.
 - Representar claramente las conexiones lógicas entre los conceptos.
 - Utilizar colores, íconos, ramas visuales y etiquetas para facilitar la lectura.
 - Incluir breves ejemplos o notas aclaratorias cuando sea pertinente.
 - Asegurarse de cubrir tanto aspectos teóricos como prácticos de Git y SVN.
- 3. **Herramientas recomendadas :**
 - Para crear el mapa mental se puede utilizar alguna de las siguientes herramientas online:
 - **MindMeister** (<https://www.mindmeister.com>): Plataforma intuitiva para crear mapas mentales interactivos y colaborativos.
 - **Coggle** (<https://coggle.it>): herramienta online sencilla y fácil de usar para crear mapas mentales visuales.
 - **Lucidchart** (<https://www.lucidchart.com>): una potente herramienta para crear diagramas y mapas mentales con opciones avanzadas de personalización.
- 4. **Entregar la Actividad :**
 - Una vez terminado el mapa mental, se debe guardar en formato PDF y subirlo a la plataforma de aprendizaje antes de la fecha límite de entrega.

Objetivo del Taller

Con esta actividad, se espera que los estudiantes:

- Comprender el funcionamiento y la utilidad de herramientas de control de versiones como Git y SVN.
- Reconocer las diferencias, ventajas y limitaciones.
- Organizar visualmente el conocimiento adquirido sobre estas herramientas fundamentales para la gestión de código.
- Reforzar habilidades de análisis, síntesis y comunicación visual en el contexto del desarrollo colaborativo de software.

3.1.5 Nombre rúbrica

- RAA1_U1_Gestión_software

- RAA2_U1_Gestión_software

3.2 Unidad

Unidad 2: Gestión de la configuración

3.2.2 Nombre actividad

- AA1. Actividad: Elaboración de un Mapa Conceptual sobre la Gestión de Cambios en el Desarrollo de Software
- AA2: Taller Grupal – Línea Base Software de Restaurante

3.2.3 Tipo de actividad

- AA1: Mapa conceptual
- AA2. Taller Grupal

3.2.4 Desarrollo actividad

AA1. Actividad: Elaboración de un Mapa Conceptual sobre la Gestión de Cambios en el Desarrollo de Software

Objetivo de la actividad:

Esta actividad tiene como propósito fortalecer la capacidad de análisis y síntesis del estudiante mediante la elaboración de un mapa conceptual que integre los conceptos clave relacionados con la gestión de cambios en proyectos de desarrollo de software. A través de esta representación visual, se busca comprender cómo se planifican, evalúan, controlan y documentan los cambios que surgen durante el ciclo de vida del software, asegurando que estos se alineen con los objetivos del proyecto y no afecten negativamente la calidad del producto.

Descripción de la actividad

El estudiante debe crear un mapa conceptual en el que se organicen de forma jerárquica y lógica los principales conceptos asociados a la gestión de cambios. Esta representación debe mostrar cómo se realiza la solicitud, evaluación, aprobación, implementación y seguimiento de los cambios en un entorno de desarrollo de software.

En el mapa conceptual deberás incluir, al menos, los siguientes elementos:

- Definición general de gestión de cambios en software.
- Importancia de gestionar adecuadamente los cambios a lo largo del ciclo de vida del desarrollo.
- Fases del proceso de gestión de cambios:
- Identificación y registro del cambio (RFC).
 - Evaluación del impacto técnico, económico y funcional.



- Aprobación o rechazo del cambio.
- Implementación controlada del cambio.
- Verificación y documentación posterior.
- Herramientas utilizadas en la gestión de cambios (por ejemplo: Git, Jira, Jenkins).
- Roles involucrados (desarrolladores, analistas, gestores de proyecto, QA).
- Diferencias entre cambios correctivos, adaptativos y evolutivos.
- Criterios para decidir si un cambio debe ser aprobado o no (alineación con objetivos, riesgos).
- Consecuencias de una mala gestión de cambios (reprocesos, pérdida de calidad, retrasos).
- Relación entre la gestión de cambios y otras prácticas como integración continua y control de versiones.

Forma de entrega:

- Realiza el mapa conceptual en formato digital (imagen o PDF).
- Subir el archivo a la plataforma virtual en el espacio habilitado para la actividad.
- Incluir el nombre completo y el título de la actividad en el documento.

AA2: Taller Grupal – Línea Base Software de Restaurante

Objetivo de la actividad

La finalidad de esta actividad es que los estudiantes apliquen los conocimientos adquiridos sobre gestión de configuración y control de versiones, mediante la definición y documentación de una línea base del código fuente para un sistema de software orientado a la gestión operativa de un restaurante. Esta línea base será utilizada como punto de referencia para el control de cambios futuros, garantizando la trazabilidad y estabilidad del desarrollo.

A través de este ejercicio, los participantes desarrollan habilidades en la documentación técnica, la colaboración grupal, y la organización de entregables base en el contexto de proyectos de software reales.

Descripción de la actividad

Taller: Establecimiento de la Línea Base Sistema de Gestión de Restaurante

Los estudiantes, organizados en grupos de máximo 3 integrantes, deberán levantar, analizar y documentar una **línea base del sistema de gestión de restaurante**, considerando sus módulos más relevantes: pedidos, reservas, gestión de menús, usuarios, reportes y facturación. La línea base incluirá tanto aspectos funcionales como

técnicos relacionados con la versión inicial estable del código fuente y sus respectivos artefactos.

Esta actividad se enfoca en definir el primer hito de estabilidad del proyecto, sobre el cual se construirá el control de versiones, la gestión de cambios y futuras validaciones.

Instrucciones para los Estudiantes:

Estructura del Documento Entregable

1. Portada

- Título del taller.
- Nombres completos de los integrantes del grupo.
- Nombre del curso.
- Fecha de entrega.

2. Introducción

Breve presentación del sistema de gestión para restaurante y justificación de la importancia de establecer una línea base del código fuente en las etapas tempranas del desarrollo.

3. Objetivo del Taller

Describir el propósito del ejercicio y las competencias que se pretenden reforzar: gestión de configuración, documentación técnica y planificación de versiones.

4. Contenido Técnico

4.1. Descripción del Proyecto

- Nombre del sistema: *Sistema Integral de Gestión de Restaurante (SIGR)*.
- Breve descripción: Aplicación web para gestionar pedidos, reservas, administración de menús, control de caja y generación de reportes.

4.2. Componentes Incluidos en la Línea Base

- Módulo de autenticación de usuarios (clientes, meseros, administrador).
- Módulo de menú digital con CRUD de platos y categorías.
- Módulo de registro y seguimiento de pedidos en tiempo real.
- Módulo de reservas por fecha y hora.
- Módulo de cierre de caja y reportes de ventas diarios.

4.3. Versionado del Código



- Herramienta utilizada: Git.
- Repositorio oficial: <https://github.com/equipoX/sigr>
- Rama principal estable: main
- Commits incluidos: desde el inicial hasta el commit identificado con el hash a93b4f1.

4.4. Criterios para Establecer la Línea Base

- Todos los módulos iniciales se compilan correctamente.
- Las funcionalidades críticas están operativas y validadas.
- Pruebas básicas de integración completadas.
- Estructura de carpetas y convenciones de código definidas.
- Documentación técnica mínima adjunta (README, estructura de base de datos, manual de despliegue).

4.5. Herramientas de Soporte

- GitHub para control de versiones.
- GitHub Issues para seguimiento de mejoras y bugs.
- Jenkins (opcional) para integración continua futura.

4.6. Documentación Asociada

- README.md: instrucciones para clonar y ejecutar el sistema.
- CHANGELOG.md: historial de cambios incluidos hasta la línea base.
- LICENSE.txt: tipo de licencia aplicada al software (MIT o GPL).

4.7. Validación y Aprobación de la Línea Base

- Fecha de creación: [dd/mm/aaaa].
- Validado por: grupo de desarrollo.
- Responsable de aprobación: coordinador del equipo o rol designado.

Formato de entrega

- Formato: Documento PDF.
- Extensión máxima: 20 páginas (sin incluir portada).
- Nombre del archivo: *Taller_LineaBase_SIGR.pdf*.
- Plataforma de envío: Subir a la sección de actividades dentro del curso virtual.

3.2.5 Nombre rúbrica

- RAA1_U2_Gestión_software
- RAA2_U2_Gestión_software



3.3 Unidad

Unidad 3: Automatización y entrega continua

3.3.2 Nombre actividad

- AA1: Estudio de Caso: Integración Continua en un Proyecto de Software
- AA2 – ABP: Proyecto Integrador – Construcción de un Pipeline para un Proyecto de Software

3.3.3 Tipo de actividad

- AA1: Estudio de Caso
- AA2. ABP: Proyecto Integrador

3.3.4 Desarrollo actividad

AA1: Estudio de Caso: Integración Continua en un Proyecto de Software

Contexto del Caso

Una empresa de desarrollo de software fue contratada para construir un sistema de pedidos y reservas en línea para un restaurante. Durante las fases iniciales del proyecto, el equipo avanzó rápidamente en la codificación y entrega de funcionalidades. Sin embargo, al llegar a las etapas de integración y prueba, comenzaron a surgir problemas significativos:

- Los desarrolladores trabajaban en ramas separadas sin una integración frecuente, lo que generó conflictos de código difíciles de resolver.
- No se aplicaron pruebas automatizadas con regularidad, lo que permitió que errores críticos llegaran a etapas avanzadas del desarrollo.
- El cliente recibió versiones inestables del sistema, lo que redujo su confianza en el proyecto.
- Se perdía tiempo corrigiendo errores que pudieron haberse detectado antes mediante automatización.

Frente a esta situación, el equipo de trabajo propone aplicar prácticas de **Integración Continua (CI)** para mejorar la calidad del desarrollo y evitar estos problemas en futuros ciclos del proyecto.

Actividades a Realizar



A partir de la lectura del caso propuesto, se solicita a los estudiantes elaborar un **informe técnico individual** que contenga el análisis y desarrollo de las siguientes preguntas orientadoras:

1. ¿Por qué es importante aplicar integración continua desde las primeras fases del desarrollo de software?
2. ¿Qué herramientas y prácticas se utilizan comúnmente para implementar la integración continua? Mencione al menos tres.
3. ¿Qué consecuencias puede tener la ausencia de integración continua en proyectos colaborativos?
4. ¿Cómo influye la integración continua en la calidad del producto final y la satisfacción del cliente?
5. Si el equipo implementara integración continua en este proyecto del sistema del restaurante, ¿qué pasos debería seguir y qué beneficios concretos obtendría?

Estructura del Informe

El informe técnico deberá estructurarse de la siguiente manera:

- **Portada:**
 - Título del informe.
 - Nombre completo del estudiante.
 - Nombre del curso.
 - Fecha de entrega.
- **Introducción:**
 - Breve presentación del caso.
 - Propósito del análisis.
- **Marco conceptual:**
 - Definición de integración continua.
 - Herramientas comunes (por ejemplo: Jenkins, GitHub Actions, GitLab CI).
 - Relación entre CI y metodologías ágiles.
- **Desarrollo del análisis:**
 - Respuestas argumentadas a las preguntas orientadoras.
 - Propuestas de solución específicas para el caso.
 - Esquemas o ejemplos opcionales para reforzar la propuesta (por ejemplo: etapas de un pipeline, roles involucrados, riesgos mitigados).



- **Conclusión:**

- Reflexión final sobre la importancia de integrar buenas prácticas de CI.
- Principales aprendizajes obtenidos.

Objetivos de la Actividad

- Comprender la relevancia de la integración continua en proyectos de software.
- Identificar herramientas y prácticas adecuadas para su implementación.
- Aplicar el análisis crítico a una situación realista de desarrollo.
- Proponer soluciones técnicas que mejoren la calidad y la colaboración.

Entregables

- **Tipo de archivo:** PDF.
- **Extensión máxima:** 15 páginas.
- **Nombre del archivo:** *AA1_EstudioCaso_IntegracionContinua.pdf*
- **Plataforma de entrega:** A través del entorno virtual de aprendizaje, en el espacio correspondiente a la actividad AA1.

AA2 – ABP: Proyecto Integrador – Construcción de un Pipeline para un Proyecto de Software

Objetivo del Proyecto

Diseñar, documentar y simular un **pipeline de integración y entrega continua (CI/CD)** para un sistema de software realista. El objetivo es integrar prácticas de automatización que permitan compilar, probar y desplegar el sistema de forma eficiente, confiable y sin intervención manual, garantizando entregas frecuentes y de calidad.

Descripción del Proyecto

Se plantea el desarrollo de un sistema de gestión de pedidos y reservas para un restaurante. Este sistema permite a los usuarios realizar reservas, consultar disponibilidad de mesas, gestionar pedidos en línea y recibir confirmaciones automáticas. Como parte del equipo de DevOps y aseguramiento de calidad, se debe construir un pipeline CI/CD que automatice el flujo de trabajo desde la integración del código hasta el despliegue en un entorno simulado o de pruebas.

El propósito de este proyecto integrador es demostrar cómo las herramientas de automatización pueden mejorar el control de calidad, reducir los errores en producción y acelerar el ciclo de vida del desarrollo de software.



Actividades del Proyecto

1. Análisis del Sistema

- Describir brevemente las funcionalidades principales del sistema de restaurante.
- Identificar los módulos que deben ser incluidos en el pipeline (por ejemplo: reservas, pedidos, autenticación de usuarios, reportes).
- Determinar los puntos críticos del flujo de desarrollo que deben ser automatizados.

2. Diseño del Pipeline CI/CD

- Definir las etapas principales del pipeline (por ejemplo: construcción, pruebas, validación, despliegue).
- Explicar qué herramientas se utilizarán para cada etapa (por ejemplo: Git para control de versiones, Jenkins o GitHub Actions para orquestación, Docker para empaquetado, etc.).
- Elaborar un diagrama del pipeline completo, indicando la secuencia y los procesos automatizados.
- Presentar un ejemplo de configuración en pseudocódigo o YAML, simulando un archivo como Jenkinsfile o .github/workflows/deploy.yml.

3. Simulación de Ejecución del Pipeline

- Describir cómo se simulará la ejecución del pipeline paso a paso.
- Indicar qué resultados se esperan en cada etapa (por ejemplo: pruebas unitarias exitosas, despliegue a un entorno de staging).
- Simular fallos en una o más etapas (por ejemplo, pruebas fallidas) y proponer medidas de corrección.
- Incluir evidencias como capturas de pantalla, comandos de consola simulados o flujos en pseudocódigo.

4. Validación de Calidad y Seguridad

- Explicar cómo se validan requisitos no funcionales (rendimiento, seguridad, disponibilidad) dentro del pipeline.
- Mencionar al menos dos herramientas o técnicas que se integren en el proceso para asegurar estas validaciones (por ejemplo: pruebas de estrés, análisis estático de código, escaneo de vulnerabilidades).

5. Buenas Prácticas y Automatización Sugerida

- Mencionar buenas prácticas implementadas en el pipeline (como revisión de código, pruebas automatizadas, despliegue controlado).
- Sugerir estrategias para escalar el pipeline a ambientes de producción o desarrollo colaborativo.



Estructura del Documento a Entregar

El archivo PDF a entregar deberá estar estructurado de la siguiente forma:

- **Portada:**
 - Título del proyecto.
 - Nombre completo del estudiante.
 - Nombre del curso.
 - Fecha de entrega.
- **Introducción:**
 - Contexto del sistema de restaurante.
 - Justificación de la necesidad de un pipeline automatizado.
- **Desarrollo:**
 - Descripción técnica del pipeline y sus etapas.
 - Herramientas utilizadas.
 - Diagrama y archivo de configuración simulado.
 - Evidencias de ejecución (simuladas o reales).
 - Validación de requisitos no funcionales.
- **Conclusiones:**
 - Reflexión sobre la importancia de los pipelines en el desarrollo moderno.
 - Aprendizajes adquiridos durante el diseño del proyecto.

Objetivos de la Actividad

- Comprender los conceptos clave de integración y entrega continua en entornos DevOps.
- Diseñar pipelines que automaticen tareas repetitivas en el desarrollo de software.
- Aplicar herramientas y buenas prácticas de automatización.
- Reconocer la relación entre CI/CD y la calidad del producto entregado.

Entregables

- **Tipo de archivo:** PDF.
- **Nombre del archivo:** *AA2_Pipeline_SistemaRestaurante.pdf*
- **Extensión sugerida:** Máximo 15 páginas (sin contar portada).
- **Plataforma de entrega:** A través del aula virtual en la sección correspondiente.

3.3.5 Nombre rúbrica



- RAA1_U3_Gestión_software
- RAA2_U3_Gestión_software

4. ORIENTACIONES METODOLÓGICAS

Resultado de aprendizaje	Unidad	Actividades	Criterio de evaluación	Evidencias
Explica la importancia del control de versiones y utiliza herramientas para gestionar los cambios en el código de proyectos de software.	1	AA1	Describe los conceptos clave del control de versiones	AA1. Foro de Discusión: Resolución de Conflictos en Git 5 %
		AA2	Implementa herramientas de control como Git	AA2. Taller: Mapa Mental sobre Herramientas de Control de Versiones (Git y SVN) 15 %
			Evalúa la efectividad del control en proyectos reales	
Administra la configuración de software, asegurando la integridad y trazabilidad de los componentes a lo largo del ciclo de vida del proyecto.	2	AA1	Identifica componentes clave en la configuración	AA1. Actividad: Elaboración de un Mapa Conceptual sobre la Gestión de Cambios en el Desarrollo de Software 5 %
		AA2	Administra cambios de forma controlada	AA2: Taller Grupal – Línea Base Software de Restaurante 20 %
			Evalúa la trazabilidad en el ciclo de vida	
Implementa prácticas de automatización y entrega continua para mejorar la eficiencia en el despliegue y mantenimiento de proyectos de software.	3	AA1	Configura pipelines de automatización	AA1: Estudio de Caso: Integración Continua en un Proyecto de Software 5 %
		AA2	Optimiza procesos de entrega continua	AA2-ABP: Proyecto Integrador – Construcción de un Pipeline para un Proyecto de Software 20 %
			Evalúa la eficiencia lograda con estas prácticas	



5. FUNDAMENTACIÓN TEMÁTICA

Unidad N°	Descripción de la Unidad
I	La unidad “Control de versiones” permite comprender el papel estratégico que desempeñan las herramientas de control en la organización, seguimiento y colaboración dentro de proyectos de desarrollo de software. A través del estudio de sistemas como Git y Subversion (SVN), el estudiante puede familiarizarse con los fundamentos del versionamiento y su aplicación en contextos reales de trabajo en equipo. Uno de los ejes centrales abordados es la creación y gestión de ramas, entendida como una técnica que permite desarrollar nuevas funcionalidades, corregir errores o experimentar sin comprometer la versión principal del código. A partir de este enfoque, también se ha explorado el proceso de fusión de ramas, reconociendo la importancia de una integración cuidadosa y ordenada de los cambios en entornos colaborativos. Asimismo, se profundiza en la resolución de conflictos, una competencia esencial cuando dos o más desarrolladores modifican simultáneamente partes del código. Mediante ejercicios prácticos, se ha demostrado cómo identificar y solucionar conflictos, manteniendo la coherencia del proyecto y previniendo errores costosos. En conjunto, los contenidos estudiados en esta unidad fortalecen la capacidad del estudiante para trabajar de manera estructurada en entornos de desarrollo colaborativo, favoreciendo la transparencia, la trazabilidad y la eficiencia en la gestión del código fuente. Estos conocimientos constituyen una base sólida para las siguientes unidades del curso y resultan indispensables en el ejercicio profesional dentro de equipos ágiles y proyectos de software de cualquier escala.
ii	La gestión de la configuración es un proceso esencial en el ciclo de vida del software, ya que permite mantener el control sobre los cambios que se realizan en los artefactos del proyecto, garantizando la coherencia, trazabilidad y calidad del producto final. Esta unidad proporciona a los estudiantes los conocimientos necesarios para planificar, controlar y documentar las modificaciones en los elementos de configuración a lo largo del desarrollo del software. Durante la unidad se exploran tres bloques principales: • Gestión de cambios, donde se abordan los tipos de cambios, su solicitud, evaluación y control; • Definición de líneas base, que establece puntos de referencia estables para el seguimiento del proyecto;



- y el **uso de herramientas como Jenkins**, aplicadas a la automatización y supervisión de los procesos de integración y entrega continua.

Al finalizar la unidad, los estudiantes serán capaces de implementar un sistema de gestión de configuración eficaz, utilizar herramientas que respalden la automatización del control de versiones y aplicar buenas prácticas que aseguren la integridad de los productos de software. Las actividades prácticas complementarán el aprendizaje con situaciones reales que fortalezcan la toma de decisiones en entornos colaborativos y de cambio constante.

III

La automatización y la entrega continua representan una evolución esencial en los procesos de desarrollo y gestión de software, permitiendo a los equipos mejorar la calidad, la velocidad y la estabilidad de los productos entregados. Esta unidad proporciona un recorrido estructurado por los fundamentos y herramientas que habilitan un flujo de trabajo eficiente y automatizado en entornos reales de desarrollo.

En primer lugar, se abordan los **fundamentos de la integración continua (CI)**, entendida como una práctica clave que permite integrar cambios de código de forma frecuente y controlada. Se revisaron herramientas como Jenkins, GitLab CI y GitHub Actions, que facilitan la automatización de compilaciones y pruebas en cada confirmación de código, mejorando la detección temprana de errores.

Posteriormente, se explora el papel de las **pruebas automáticas dentro de los pipelines CI/CD**, destacando su importancia para validar la funcionalidad del software en cada etapa del ciclo de vida. Se analizaron tipos de pruebas, herramientas como JUnit, Selenium y SonarQube, así como la gestión de reportes y métricas de calidad en entornos automatizados.

Finalmente, se estudia la **implementación de DevOps**, un enfoque que promueve la colaboración entre equipos de desarrollo y operaciones para lograr una entrega más rápida y segura. Se profundizó en sus componentes culturales y técnicos, sus retos comunes y las mejores prácticas para su adopción efectiva.

Todo lo aprendido en esta unidad está orientado a cumplir el resultado de aprendizaje de diseñar e implementar flujos automatizados que integren pruebas, validaciones y despliegues continuos, fortaleciendo así la agilidad y confiabilidad del software desarrollado. Dominar estos conceptos permite a los futuros profesionales enfrentarse con solidez a los desafíos de la entrega continua, optimizando procesos y mejorando la experiencia del usuario final.